

Classification 1

Classification in machine learning is a type of supervised learning where the goal is to predict the category or class of an input based on its features

Types of classification

[1] **Binary classification** : involves two classes or categories , Examples : Email classification , Disease diagnosis, Gender Classification

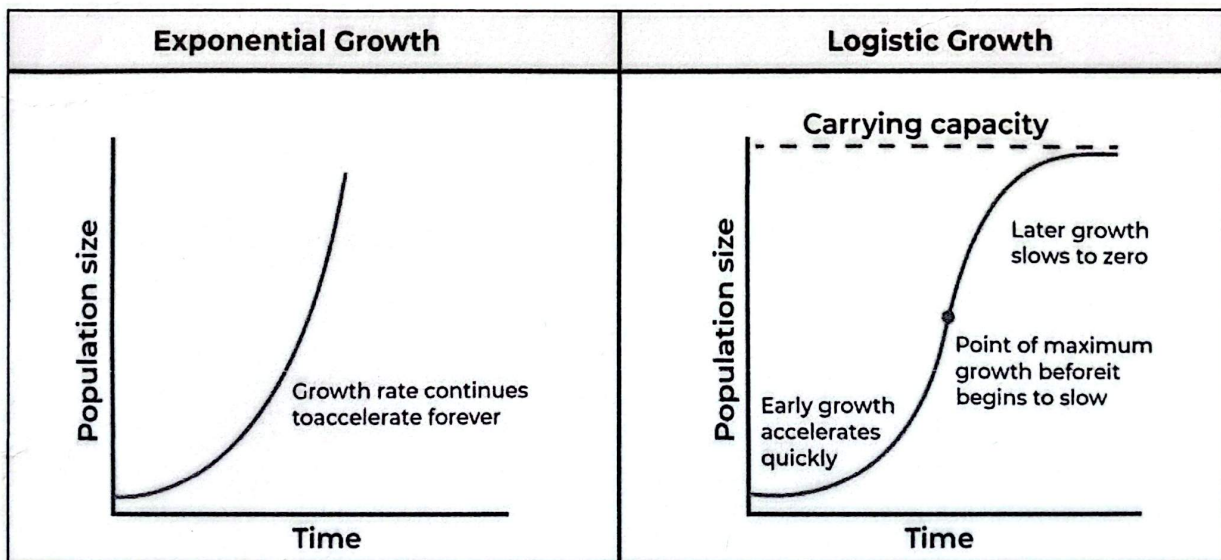
[2] **Multiclass classification** : more than two classes, Each input is classified into one of multiple possible categories, Examples : Handwriting recognition , image classification

[3] **Multilabel classification** : each instance can be assigned multiple labels simultaneously, Examples : movie tagging (where movie might be labeled with (action), (comedy), and (adventure))

standard logistic probability distribution function

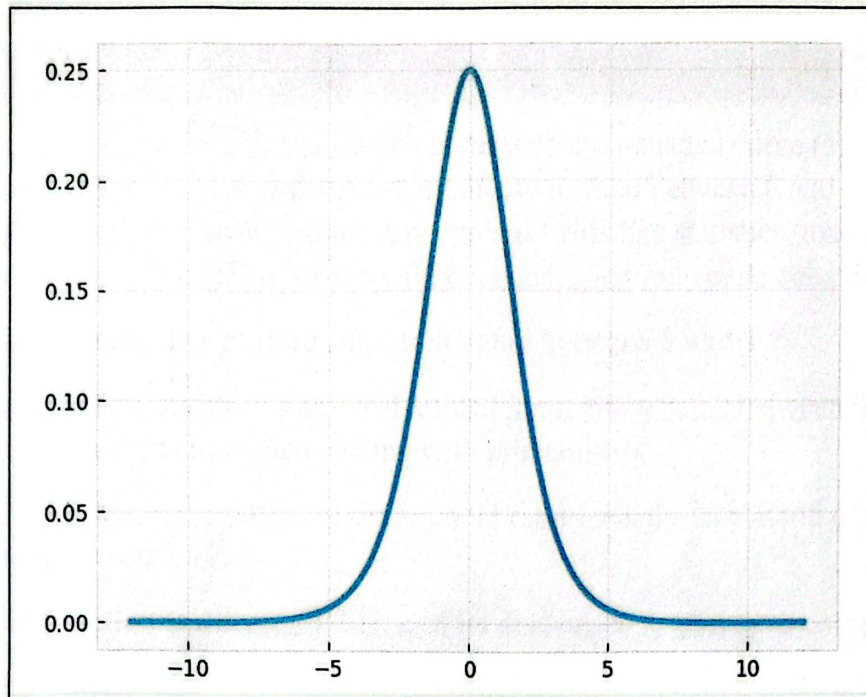
The standard logistic distribution is a continuous probability distribution that is frequently used in statistics, particularly in logistic regression and neural network.

The standard logistic distribution has a similar shape to the normal distribution, but it has heavier tails, which means that extreme values are more likely.



Probability distribution function of Standard logistic distribution is :

$$f(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{(e^{\frac{x}{2}} + e^{-\frac{x}{2}})^2} = \frac{1}{4} \operatorname{sech}^2\left(\frac{x}{2}\right)$$

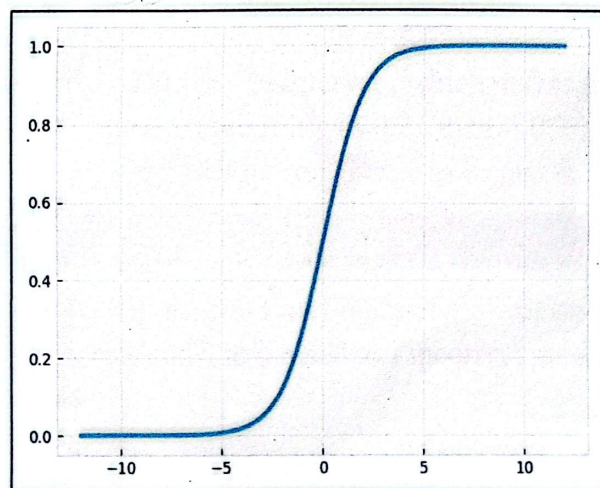


To find the cumulative distribution function, we can integrate $f(x)$ as below :

$$\begin{aligned}
 F(X) &= \int_{-\infty}^x \frac{e^{-x}}{(1+e^{-x})^2} dx = - \int_{-\infty}^x \frac{de^{-x}}{(1+e^{-x})^2} = - \int_{-\infty}^x (1+u)^{-2} du \\
 &= \frac{-1}{-1} [(1+u)^{-1}]_{-\infty}^x = [(1+e^{-x})^{-1}]_{-\infty}^x \\
 &= \frac{1}{1+e^{-x}} - \frac{1}{1+\infty} = \frac{1}{1+e^{-x}}
 \end{aligned}$$

So CDF $F(x)$ of standard logistic distribution in final formula is :

$$F(X) = \frac{1}{1+e^{-x}}$$



Sigmoid function $\sigma(x)$

The sigmoid function is a mathematical function with an S-shaped curve (also called a sigmoid curve) that maps any real-valued number to a value between 0 and 1. It is widely used in mathematics, probability, and many applied fields like statistics, machine learning, and biology because of its smooth, bounded, and predictable behavior.

Why we need to map any real number to a value between 0 and 1 ?

The sigmoid function transforms any real-valued input into a value between 0 and 1, making it particularly useful when dealing with probabilities

In binary classification, the output of the sigmoid represents the likelihood of a data point belonging to the positive class.

The sigmoid function is differentiable, and its derivative is simple to compute:

$$\frac{d}{dx} \left(\frac{1}{1+e^{-x}} \right) = \frac{-(-e^{-x})}{(1+e^{-x})^2} = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \times \frac{e^{-x}}{1+e^{-x}} = \frac{1}{1+e^{-x}} \times \left(1 - \frac{1}{1+e^{-x}} \right) = \sigma(x)(1-\sigma(x))$$

Binary linear classification

- Predict a binary target $t \in \{0, 1\}$, Training examples with $t = 1$ are called positive examples, and training examples with $t = 0$ are called negative examples
- $t \in \{0, 1\}$ or $t \in \{-1, +1\}$ is for computational convenience.
- model prediction y is a linear function of x , followed by a threshold r

$$Z = \theta^T X + b, \quad y = \begin{cases} 1 & \text{if } z \geq r \\ 0 & \text{if } z < r \end{cases}$$

This is show how sigmoid function works

[1] sigmoid function takes an input x , applies an exponential transformation, and then maps the result to be between 0 and 1

[2] very low positive values of x will approaches e^{-x} to zero, so $\sigma(x)$ will approaches 1

[3] when $x=0$, $\sigma(x) = \frac{1}{2}$

[4] when very large negative number, this will approaches e^{-x} to ∞ , so $\sigma(x)$ will approaches 0

In binary classification, $\sigma(x)$ is in $[0, 1]$ interval, which makes the output one of the two classes, if it isn't 0 or 1, as example if it was 0.7, then if $0.7 \geq r$ then it will be considered as a class 1, else then it will be considered as a class 0.

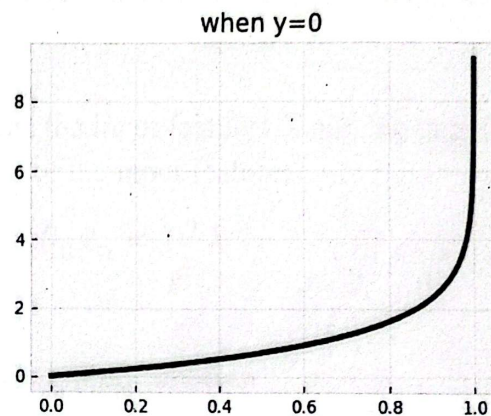
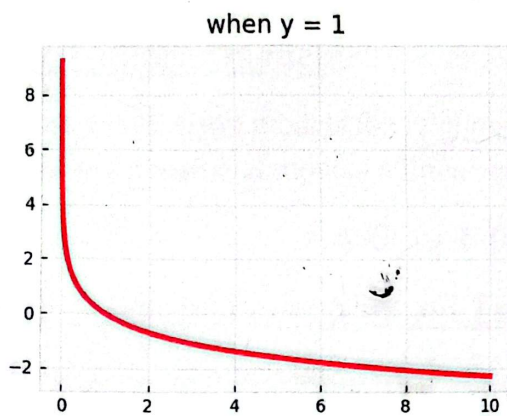
Log Loss (Binary Cross-Entropy Loss)

This is the most widely used cost function for binary classification. It is derived from the likelihood of the data given the model and penalizes incorrect predictions more as they diverge from the true labels.

$$\log \text{loss} = \begin{cases} -\frac{1}{m} \sum_{i=1}^m (1-y) \log(1-h_{\theta}(x)) & \text{when } y=0 \\ -\frac{1}{m} \sum_{i=1}^m y \log(h_{\theta}(x)) & \text{when } y=1 \end{cases}$$

$$\log \text{loss} = -\frac{1}{m} \sum_{i=1}^m [y \log(h_{\theta}(x)) + (1-y) \log(1-h_{\theta}(x))]$$

$$\text{Where } h_{\theta}(x) = \sigma(Z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-\theta^T x}}$$



These two figures show relation between loss function and predicted value, when $y=1$ you notice that when predicted value get more closer to larger values the loss function decreases, and same way for $y=0$ case

Derivative of log loss (Binary Cross-Entropy Loss)

$$\frac{\partial J}{\partial \theta_j} = \frac{\partial J}{\partial h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial \theta_j}$$

$$J = -y \log(h) - (1-y) \log(1-h)$$

$$h = \frac{1}{1+e^{-z}}$$

$$z = \theta_0 x_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

$$\frac{\partial J}{\partial \theta_j} = \left(-\frac{y}{h} + \frac{1-y}{1-h} \right) \times (h \times (1-h)) \times (x_j)$$

$$= (-y(1 - h) + h(1 - y)) \times x_j = (-y + yh + h + yh) \times x_j$$

$$\frac{\partial J}{\partial \theta_j} = (h - y) \times x_j$$

Logistic Regression

Logistic regression is a fundamental algorithm for binary classification. Unlike linear regression, which predicts continuous values, logistic regression predicts probabilities. It transforms the input using the sigmoid function to produce an output between 0 and 1, which can then be thresholded to classify data into two categories.

It works like linear regression work flow, first there is initial weights, and second we will use Gradient descent to update these weights to get the best weights for the model

Step 1 : linear model

Logistic regression models the relationship between the input features X and the target variable Y . It first computes a **linear combination** of the input features

$$z = \theta_0 x_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T X$$

Step 2: Applying the Sigmoid Function

Since we need probabilities, we pass z through the **sigmoid function**

$$\sigma(Z) = \frac{1}{1 + e^{-z}}$$

Step 3: output – input relationship

The output y represents the probability that the given input belongs to class 1 is :

$$y = P(y = 1|x) = \sigma(Z) = \frac{1}{1 + e^{-\theta^T X}}$$

The output y represents the probability that the given input belongs to class 0 is :

$$y = P(y = 0|x) = \sigma(Z) = \frac{1}{1 + e^{-\theta^T X}}$$

Step 4 : Gradient Descent and optimization

To minimize the cost function, we use **gradient descent**, which updates the parameters iteratively.

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} = \theta_j - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x) - y)x_j$$

You can use SGD (variation of gradient descent that updates the weights one sample at a time rather than using the entire dataset) too in logistic Regression

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} = \theta_j - \alpha (h_{\theta}(x) - y)x_j$$

X1	X2	X3	Y
70	18	1	1
88	22	0	1
91	38	0	1
65	21	1	0
79	25	1	0

Assume $\theta_0, \theta_1, \theta_2, \theta_3 = 0.5, \alpha = 0.001$

X1	X2	X3	Y	Sigmoid	h-y
70	18	1	1	1	0
88	22	0	1	1	0
91	38	0	1	1	0
65	21	1	0	1	1
79	25	1	0	1	1

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x) - y)x_0 = 0.5 - 0.001 \times \frac{(2)}{5} = 0.4996$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x) - y)x_1 = 0.5 - 0.001 \times \frac{(144)}{5} = 0.4712$$

$$\theta_2 = \theta_2 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x) - y)x_2 = 0.5 - 0.001 \times \frac{(46)}{5} = 0.4908$$

$$\theta_3 = \theta_3 - \alpha \frac{1}{m} \sum_{i=0}^m (h_{\theta}(x) - y)x_3 = 0.5 - 0.001 \times \frac{(2)}{5} = 0.4996$$

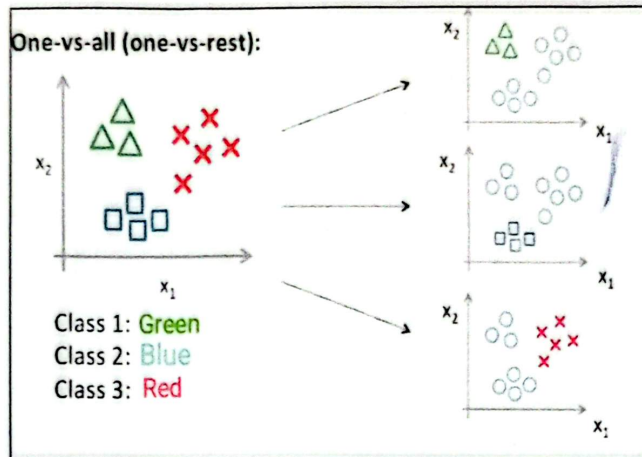
$$\nabla J = \frac{1}{m} X^T (h - y)$$

And repeat until convergence

$$\frac{1}{5} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 70 & 88 & 91 & 65 & 79 \\ 18 & 22 & 38 & 21 & 25 \\ 1 & 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Multi-classification

[1] OVA (one-vs-All)



For each class in the dataset, a separate binary classifier is trained, each classifier is responsible for distinguishing one class from all the other classes combined.

Example : suppose we have a classification problem with three classes (Cat) (Dog) (Rabbit) , and we used OVA

Step 1: Define Binary Classifiers

Classifier 1 | C1: Distinguishes (Cat) from (Dog) and (Rabbit) | Cat Vs Dog & Rabbit

Classifier 2 | C2 : Distinguishes (Dog) from (Cat) and (Rabbit) | Dog Vs Cat & Rabbit

Classifier 3 | C3 : Distinguishes (Rabbit) from (Cat) and (Dog) | Rabbit Vs Cat & Dog

Step 2: Train Each Classifier

Assign **label 1** to all training samples belonging to main class (positive class).

Assign **label 0** to all training samples belonging to other classes (negative class)

Step 3: Predict for a New Input

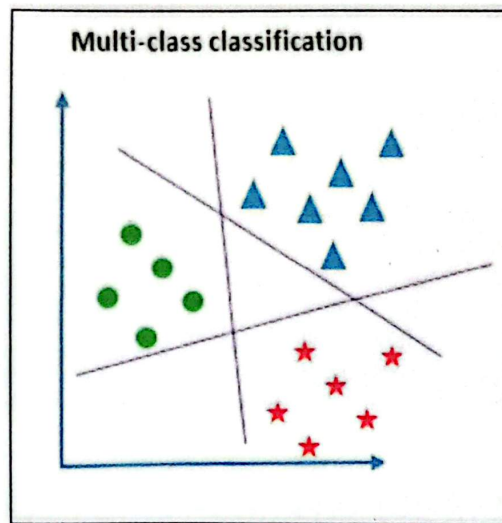
When we want to classify a new image , say an image of a (Dog)

Classifier 1 : $P(C1 | Dog) = 0.2$, Classifier 2 : $P(C2 | Dog) = 0.8$, Classifier 3 :

$P(C3 | Rabbit) = 0.1$, so highest probability for Classifier 2 where the Dog is 1 and all remain classifiers are 0

$$y = \operatorname{argmax}(P(C_k|x))$$

[2] OVO (One-Vs-One)



Trains one binary classifier for every pair of classes

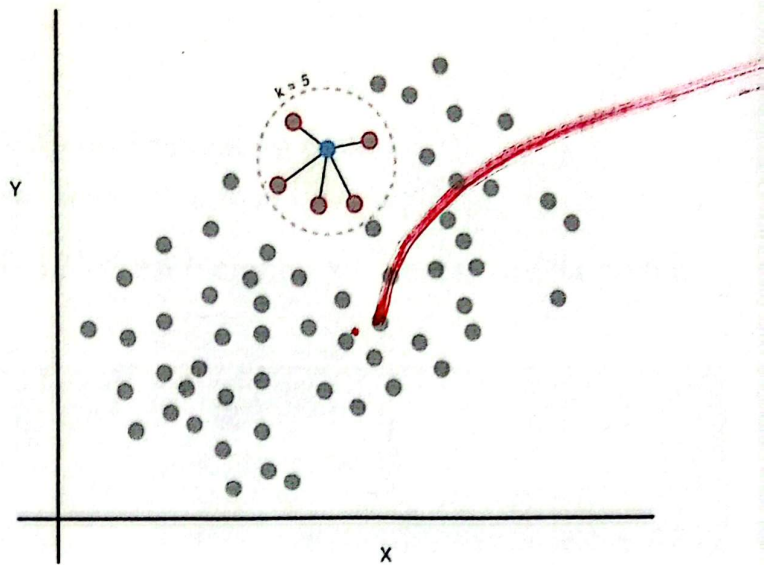
For each pair only the two classes in the pair are compared.

$$\text{number of classifiers} = \frac{n(n-1)}{2}$$

	OVA	OVO
Pros	1- easy to implement 2- number of classifiers require Is equal to the number of classe 3- Efficient for Large N 4- works well with probabilistic classifiers	1- Better accuracy than OVA for Small N 2- only two classes at time which Can reduce the impact of class imbalance
Cons	1- sometimes it is biased 2- complex 3- performance can be degrade If one class is highly imbalance or if classifiers don't generalize well	1- training and maintaining a lar Number of classifiers can be Resource- intensive and time consuming
Use case	In any case	Low accuracy when and long Training time when N is large

K-Nearest Neighbors (KNN)

- KNN is a simple intuitive algorithm used for both classification and regression
- KNN doesn't explicitly train a model in the traditional sense , Instead it stores the entire training dataset
- It compute distances between the new datapoint and all points in the training dataset.
- Common distance metrics include [Euclidean , Manhattan , and Minkowski]



Euclidean distance

For two points $(x_1, x_2, x_3, x_4, \dots, x_n)$ and $(y_1, y_2, y_3, \dots, y_n)$ in n-dimensional space, the Euclidean distance d is calculated as :

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Most commonly used distance metric, and it represents the shortest path between two points, similar to measuring the length of a line segment

Manhattan distance

For two points $(x_1, x_2, x_3, x_4, \dots, x_n)$ and $(y_1, y_2, y_3, \dots, y_n)$ in n-dimensional space, the Manhattan distance d is calculated as :

$$d = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n| = \sum_{i=1}^n |x_i - y_i|$$

It measures the distance by summing the absolute differences along each dimension, effectively creating a grid-like path

Minkowski distance

For two points $(x_1, x_2, x_3, x_4, \dots, x_n)$ and $(y_1, y_2, y_3, \dots, y_n)$ in n-dimensional space, the Minkowski distance d with parameter (p) is calculated as :

$$d = (|x_1 - y_1|^p + |x_2 - y_2|^p + \dots + |x_n - y_n|^p)^{\frac{1}{p}} = \left(\sum_{i=1}^p |x_n - y_n|^p \right)^{\frac{1}{p}}$$

- When $p=1$, Mankowski distance is the same as Manhattan distance
- When $p=2$, Mankowski distance is the same as Euclidean distance

In KNN , after calculate the distance, we find Nearest Neighbors (closest K data points based on the computed distances)

Classification	Regression
New data point is assigned the most Common classes among its K nearest Neighbors	It is assigned the average value Of the k nearest Neighbors

Cons of KNN

- [1] Can be slow with large datasets, as distance computations can be expensive
- [2] Not well-suited for high dimensional data

KNN can be quite effective for many applications, especially when combined with dimensionality reduction techniques like PCA

```
hpCodeRunnerFile.py > ...
1 from sklearn.neighbors import KNeighborsClassifier #classification
2 from sklearn.neighbors import KNeighborsRegressor #Regression
3
4 model = KNeighborsClassifier(n_neighbors=20)
5 model.fit()
6
```

How to choose the best K

- Too small K (e.g., $K=1$, $K=3$) $\rightarrow$ High variance, sensitive to noise (overfitting).
- Too large K (e.g., $K=100$) $\rightarrow$ High bias, smooth decision boundary (underfitting).

[1] Method 1 : Using Heuristics (Rule of Thumb)

$$K \approx \sqrt{N}$$

Note that : This rule doesn't always guarantee optimal performance

```

tempCodeRunnerFile.py > ...
1 import numpy as np
2 from sklearn.neighbors import KNeighborsClassifier #classification
3 from sklearn.neighbors import KNeighborsRegressor #Regression
4 x = []
5 y = []
6 model = KNeighborsClassifier(n_neighbors=np.sqrt(len(x)))
7 model.fit()
8

```

[2] Method 2 : using Graph for different K values

```

tempCodeRunnerFile.py > ...
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.neighbors import KNeighborsClassifier
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.decomposition import PCA
7 from sklearn.model_selection import train_test_split
8
9 data = pd.read_csv("D:/banana_quality.csv")
10 x , y = data.iloc[:, :7] , data.iloc[:, 7]
11 scalar , pca = StandardScaler() , PCA(n_components=2)
12 x = pd.DataFrame( pca.fit_transform( scalar.fit_transform(x) ) )
13 x_train , x_test , y_train , y_test = train_test_split(x,y,train_size=0.8)
14 scores_train = []
15 scores_test = []
16
17 for i in range(10,151) :
18     model = KNeighborsClassifier(n_neighbors=i)
19     model.fit(x_train,y_train)
20     scores_train.append(model.score(x_train,y_train))
21     scores_test.append(model.score(x_test,y_test))
22     print(f"Done for K={i}")
23
24
25 plt.style.use("bmh")
26 plt.figure(figsize=(15,4))
27 plt.subplot(1,2,1)
28 plt.plot(np.arange(10,151,1),scores_train,color="red")
29 plt.title("train_scores")
30 plt.subplot(1,2,2)
31 plt.plot(np.arange(10,151,1),scores_test,color="black")
32 plt.title("test_scores")
33 plt.show()
34

```

